



www.hoasen.edu.vn

BÀI GIẢNG TUẦN 9 + 10

- Con trỏ (pointer)
 - Khái niệm
 - Khai báo
 - Vùng trỏ tới
 - Cấp phát và giải phóng vùng nhớ
 - Con trỏ và mảng 1 chiều
 - Con trỏ và cấu trúc dữ liệu
- Chuỗi (string)
 - Khái niệm
 - Khai báo
 - Nhập/Xuất chuỗi
 - Truy xuất các ký tự của chuỗi
 - Các hàm xử lý chuỗi thông dụng

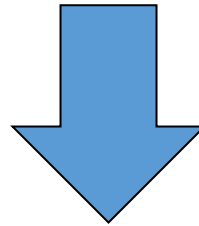
CON TRỎ?

BÀI 9

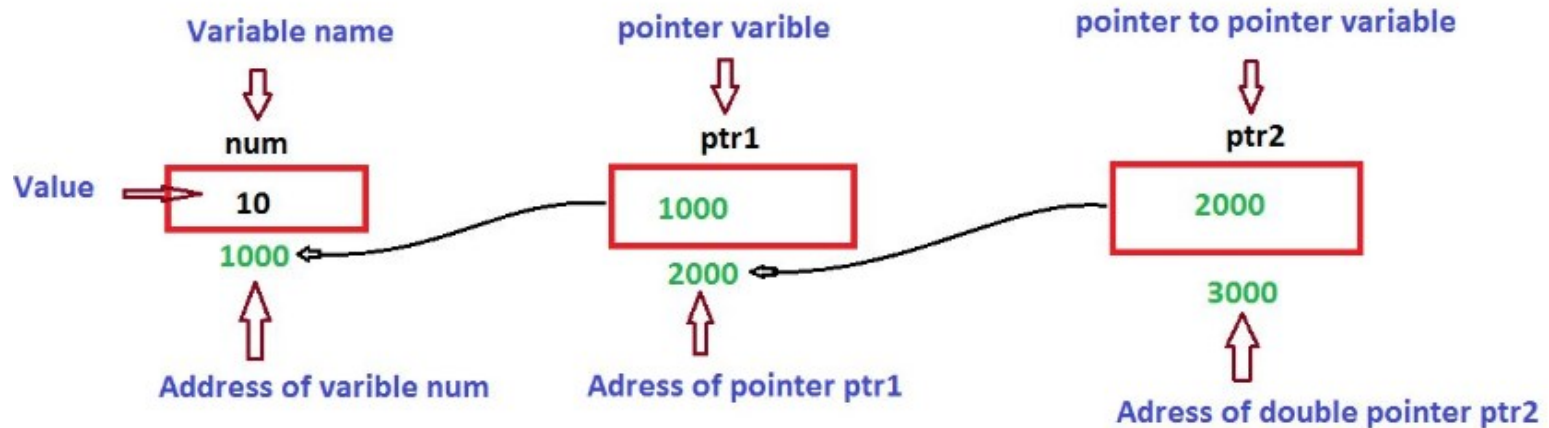
Khái niệm

Biến tĩnh là biến có kích thước, kiểu của biến và địa chỉ của nó là không đổi. Các biến này tồn tại trong suốt quá trình chạy chương trình

Biến động là loại biến có thể thay đổi được kích thước, vùng nhớ và địa chỉ của vùng nhớ được cấp phát lúc chạy chương trình. Có thể giải phóng biến động sau khi sử dụng để tiết kiệm bộ nhớ.



Biến con trỏ là loại biến dùng để chứa địa chỉ của biến động, giúp ta truy cập đến biến động



Nguồn: <https://stackoverflow.com>

Khai báo biến con trỏ

Vì con trỏ cũng là một biến nên trước khi sử dụng phải khai báo theo cú pháp sau:

Kiểu dữ liệu* Tên con trỏ;

Ví dụ:

`int* px;`

`float* m;`

`char* s;`

Vùng trỏ tới

Định vị con trỏ trỏ tới địa chỉ của một biến

Têncontrỏ = &biến

Ví dụ:

```
int a, b;  
int* px, *c;  
float* m;
```

Qui định vùng trỏ tới:

```
px = &a; //Gán địa chỉ của a cho con trỏ px  
c = &b; //Gán địa chỉ của b cho con trỏ c  
m = &a; //m là con trỏ kiểu float nên chỉ chứa  
địa chỉ biến kiểu float → không chấp nhận
```

Ví dụ

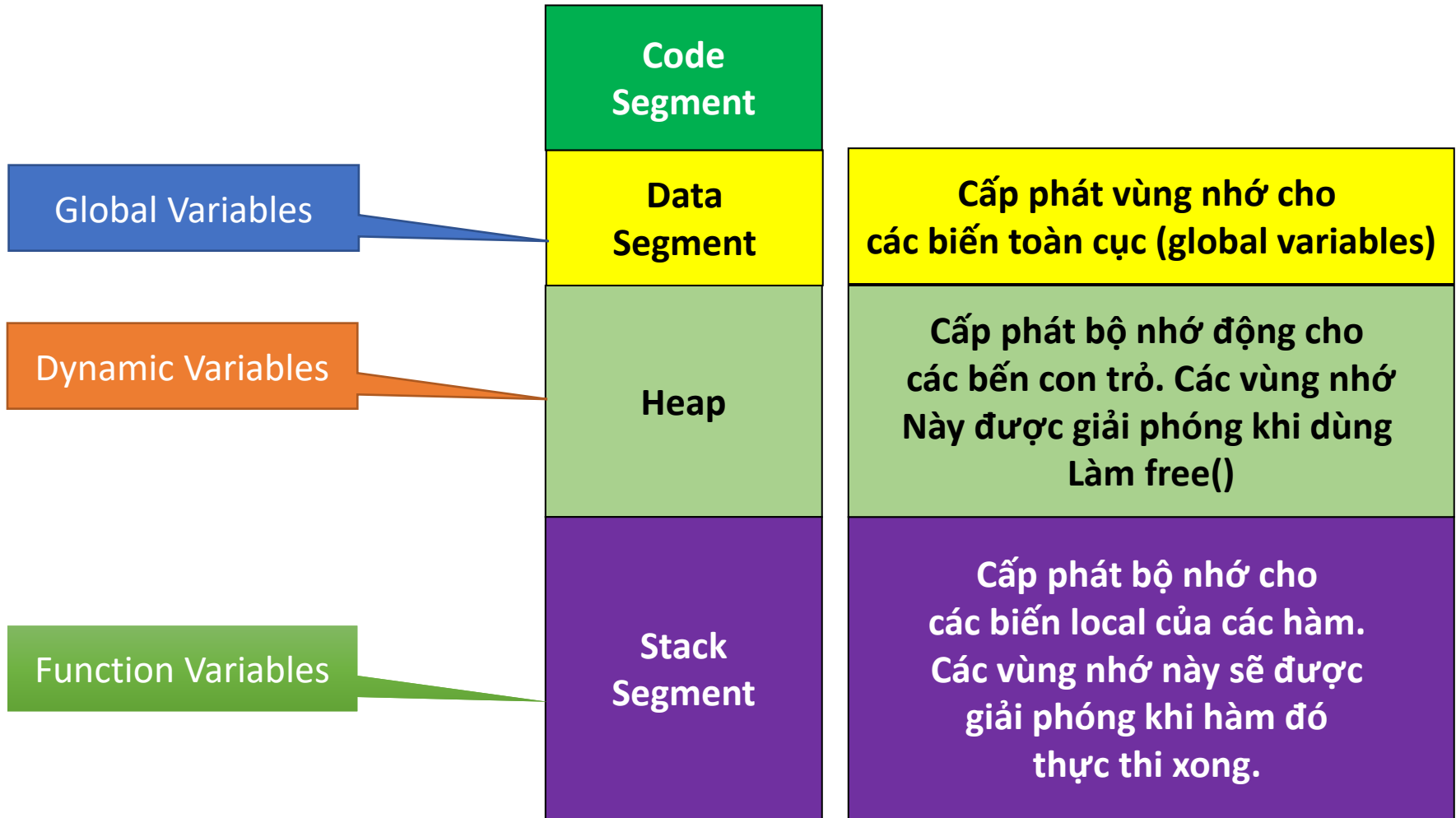
```
1  #include <stdio.h>
2  int main()
3  {
4      int N=10;
5      int *p;
6      p = &N; //p tro vào địa chỉ của N
7      printf("Gia tri cua bien N: %d\n",N);
8      printf("Dia chi cua bien N:%d\n",&N);
9      printf("Dia chi cua con tro p:%d\n",&p);
10     printf("Truy xuat gia tri cua N thong qua p:%d\n",*p);
11     *p=100; //thay doi gia tri cua N thong qua p
12     printf("Gia tri cua bien N: %d\n",N);
13     return 0;
14 }
```



```
D:\FPT\C\Pointer.exe

Gia tri cua bien N: 10
Dia chi cua bien N:2358860
Dia chi cua con tro p:2358848
Truy xuat gia tri cua N thong qua p:10
Gia tri cua bien N: 100
-----
```


Bộ nhớ được tổ chức thể nào



Bộ nhớ được tổ chức thế nào

```

1  #include <stdio.h>
2
3  int Max = 100;
4
5  int Tong(int a, int b)
6  {
7      int T;
8      printf("Dia chi bien a: %d\n",&a);
9      printf("Dia chi bien b: %d\n",&b);
10     printf("Dia chi bien T: %d\n",&T);
11     T = a+b;
12     return T;
13 }
14 int main()
15 {
16     int x=5, y=10;
17     int *p = new int;
18     printf("Dia chi bien x: %d\n",&x);
19     printf("Dia chi bien y: %d\n",&y);
20     printf("Dia chi bien p: %d\n",&p);
21     printf("Dia chi main(): %d\n",&main);
22     printf("Dia chi Tong(): %d\n",&Tong);
23     printf("Dia chi Max : %d\n",&Max);
24     printf("Tong cua x, y = %d\n",Tong(x,y));
25 }

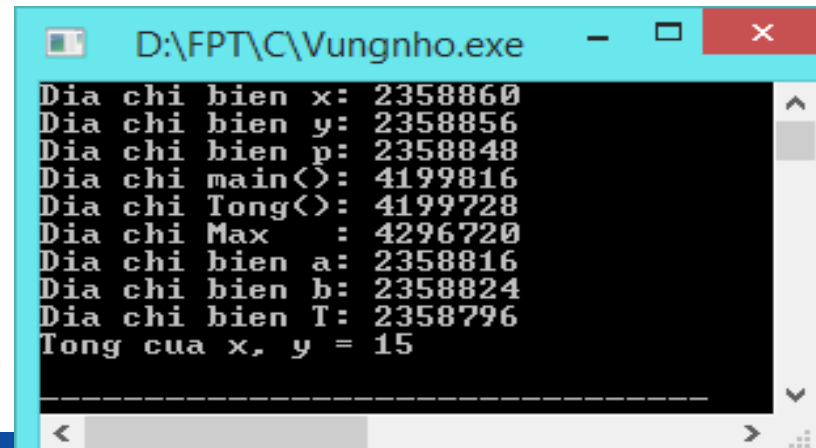
```

Data segment:
Max = 100

Code segment:
- Code of Tong
- Code of main()

Stack segment:
a, b, T,
x, y

Heap: p



```

D:\FPT\C\Vungnho.exe
Dia chi bien x: 2358860
Dia chi bien y: 2358856
Dia chi bien p: 2358848
Dia chi main(): 4199816
Dia chi Tong(): 4199728
Dia chi Max : 4296720
Dia chi bien a: 2358816
Dia chi bien b: 2358824
Dia chi bien T: 2358796
Tong cua x, y = 15

```

STACK vs HEAP

Stack	Heap
Vùng nhớ được cấp phát khi chương trình được biên dịch.	Vùng nhớ được cấp phát khi chạy chương trình (run-time).
Vùng nhớ stack được sử dụng cho việc thực thi thread. Khi gọi hàm, các biến cục bộ của hàm được lưu trữ vào block của stack (theo kiểu LIFO). Cho đến khi hàm trả về giá trị, block này sẽ được xóa tự động.	Vùng nhớ heap được dùng cho cấp phát bộ nhớ động (malloc(), new()). Vùng nhớ được cấp phát tồn tại đến khi giải phóng vùng nhớ bằng lệnh free() hoặc delete.
Kích thước vùng nhớ stack được fix cố định. Chúng ta không thể tăng hoặc giảm kích thước vùng nhớ stack. Nếu không đủ vùng nhớ stack, gây ra stack overflow. Hiện tượng này xảy ra khi nhiều hàm lồng nhau hoặc đệ quy nhiều lần dẫn đến không đủ vùng nhớ.	Khi kích thước vùng nhớ heap không đủ cho yêu cầu malloc(), new. Hệ điều hành sẽ có cơ chế tăng kích thước vùng nhớ heap.

Nguồn: <http://vncoding.net>

Cấp phát và giải phóng vùng nhớ

* Cấp phát vùng nhớ:

Thông thường, chúng ta chỉ quan tâm đến khai báo và cung cấp vùng nhớ để lưu địa chỉ của biến mà không quan tâm đến việc cung cấp vùng nhớ để lưu dữ liệu.

Cú pháp :

Biến con trỏ = malloc(sizeof(tên kiểu));

Biến con trỏ = calloc(n, sizeof(tên kiểu));

* Giải phóng vùng nhớ:

free(tên con trỏ);

Cấp phát và giải phóng vùng nhớ

Ví dụ:

- Khai báo và cấp phát vùng nhớ cho biến con trỏ N:

```
int *N = (int*)malloc(sizeof(int));
```

- Khai báo và cấp phát vùng nhớ cho biến trỏ A là một mảng có 20 phần tử:

```
int *A = (int*)malloc(20 * sizeof(int));
```

hoặc

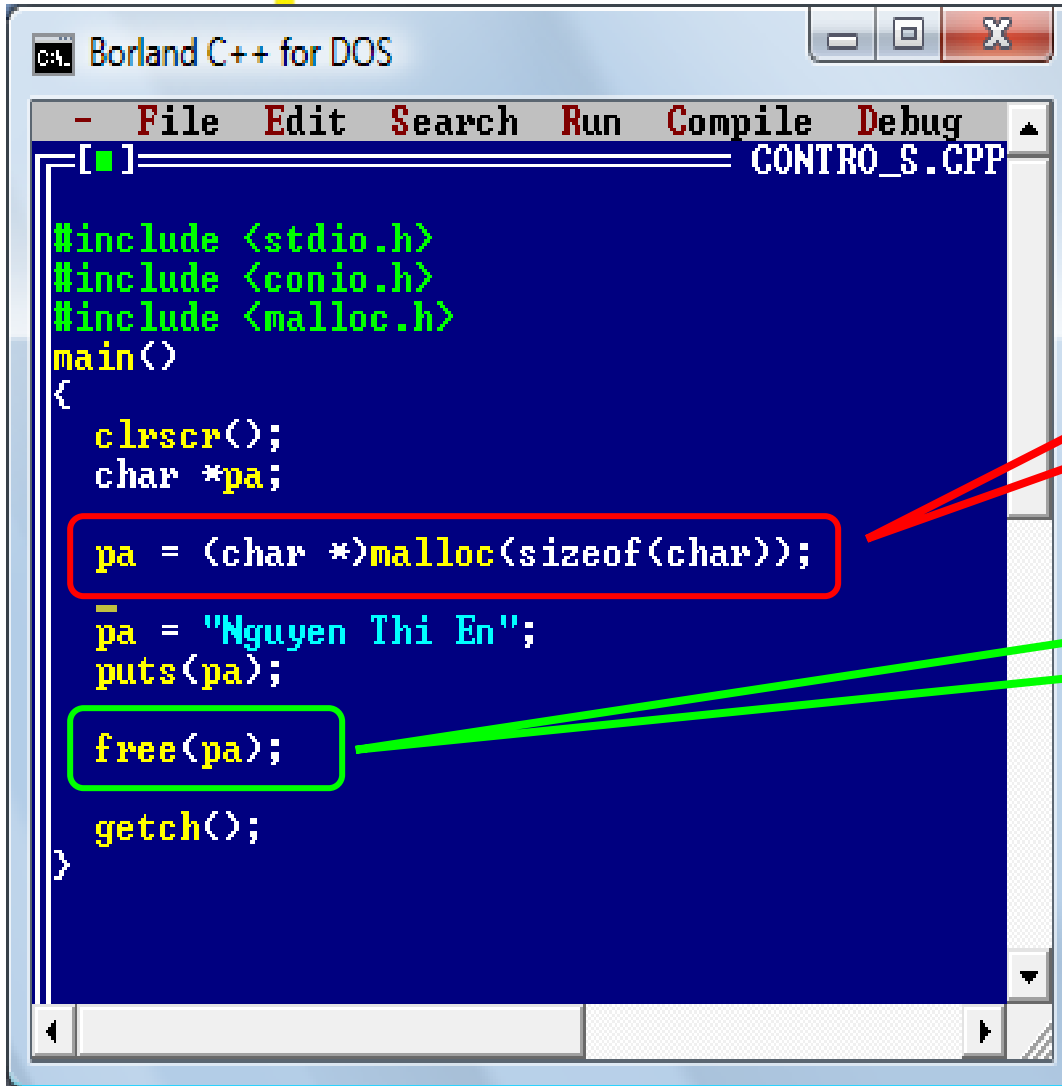
```
int *A = (int*)calloc(20, sizeof(int));
```

- Giải phóng vùng nhớ:

```
free(N);
```

```
free(A);
```

Cấp phát và giải phóng vùng nhớ



```

Borland C++ for DOS
- File Edit Search Run Compile Debug
[■] CONTRO_S.CPP

#include <stdio.h>
#include <conio.h>
#include <malloc.h>
main()
{
    clrscr();
    char *pa;
    pa = (char *)malloc(sizeof(char));
    pa = "Nguyen Thi En";
    puts(pa);
    free(pa);
    getch();
}
```

Cấp phát
vùng nhớ

Giải phóng
vùng nhớ

Cấp phát và giải phóng vùng nhớ

Do lập trình trên môi trường C++ nên có thể dùng từ khoá **new** và hàm **delete** để cấp phát và giải phóng vùng nhớ.

- Khai báo:

```
int *N = new int;
```

```
int *A = new int[20];
```

- Giải phóng vùng nhớ:

```
delete (N);
```

```
delete (A);
```



Đơn giản quá

Con trỏ và mảng 1 chiều

- Phép toán lấy địa chỉ của các phần tử trong mảng:

Ví dụ: `float a[10];`

Phép toán `&a[i]` sẽ cho địa chỉ của phần tử `a[i]`,
với `i` trong khoảng `[0,9]`

- Cách lấy giá trị của phần tử trong mảng

Ví dụ: `float a[10], *p;`

`p = a;`

→ Các cách viết sau đây là tương đương nhau:

`a[i]` `*(a+i)` `*(p+i)` `p[i]`

Con trỏ và mảng 1 chiều

GIẢI THÍCH THÊM

	a[0]	a[1]	a[2]	a[3]	a[4]
	*(a+0)	*(a+1)	*(a+2)	*(a+3)	*(a+4)
a	20	30	40	50	60
	65516	65518	65520	65522	65524

*(a+0)	value at 65516	20	a[0]
*(a+1)	value at 65518	30	a[1]
*(a+2)	value at 65520	40	a[2]
*(a+3)	value at 65522	50	a[3]

-
 -
 *(a+i) (Or) a[i]

Khai báo mảng 1 chiều và con trỏ

- Thông thường ta khai báo:

```
int a[20];
```

```
float b[20];
```

```
double c[20];
```

- Ta khai báo kiểu con trỏ như sau:

```
int *a;
```

```
char *ptr = &array_name;
```

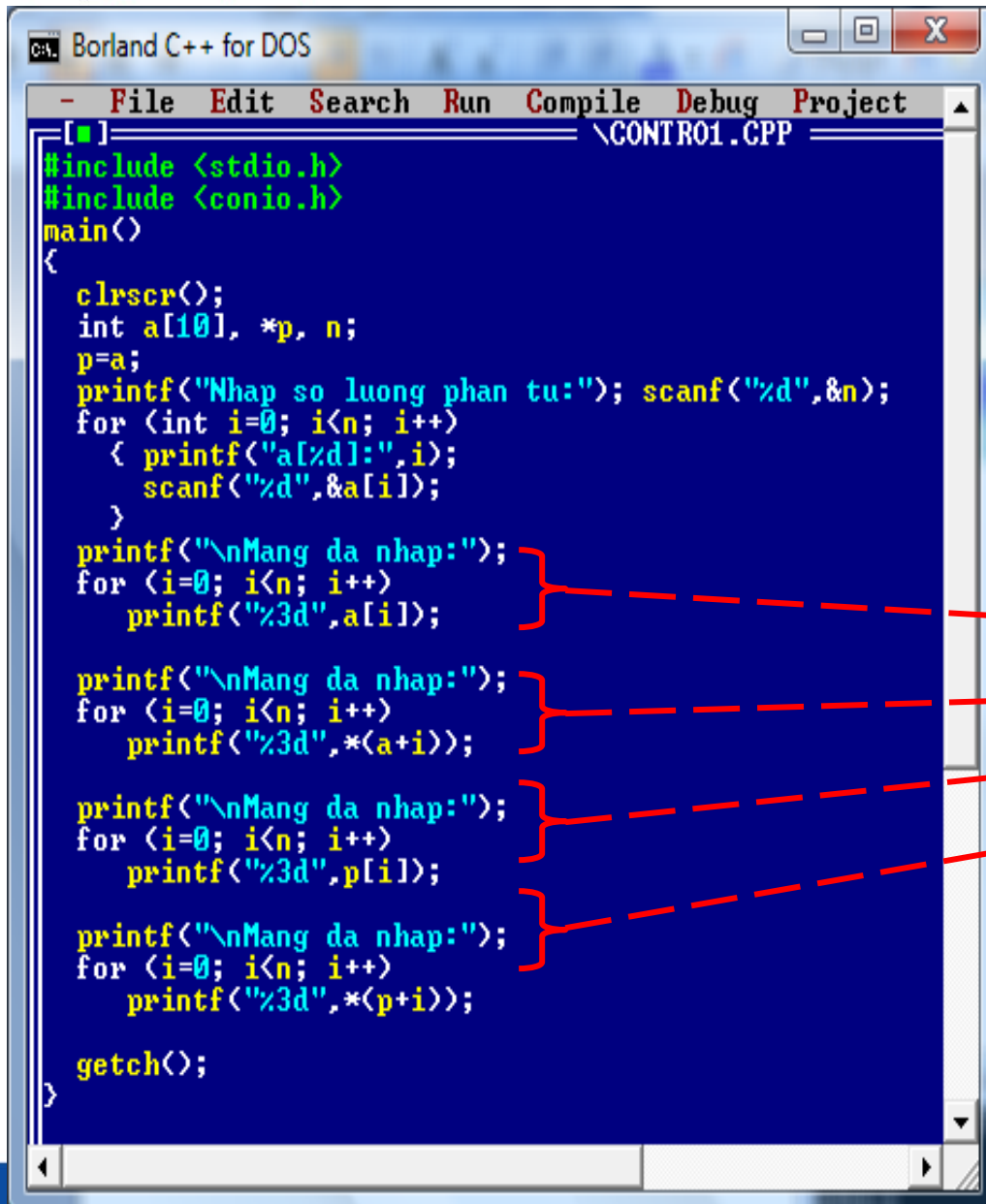
```
float *b;
```

```
double *c;
```

```
int func(int A[]);
```

```
int func(int *A);
```

Truy xuất các phần tử trong mảng



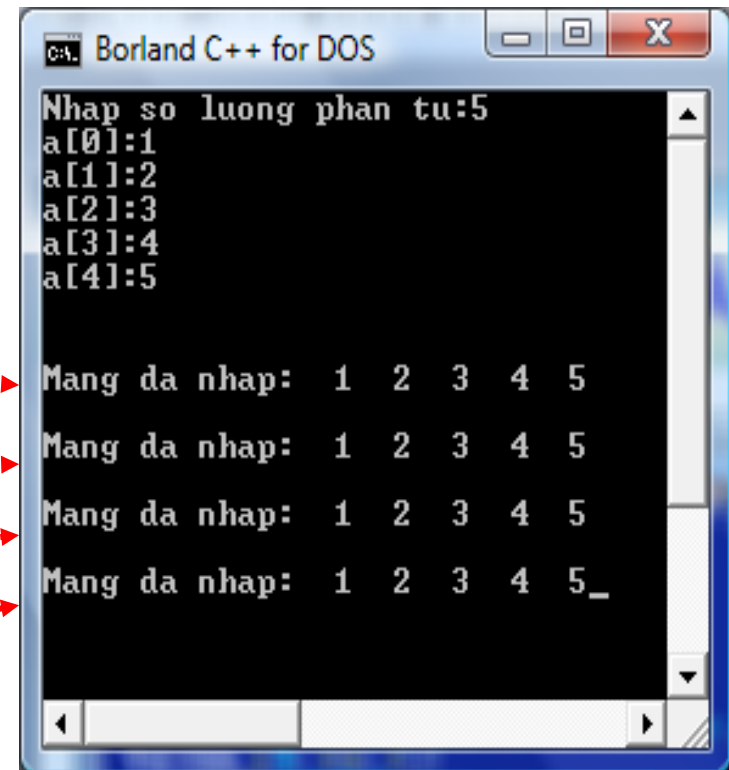
```
#include <stdio.h>
#include <conio.h>
main()
{
    clrscr();
    int a[10], *p, n;
    p=a;
    printf("Nhap so luong phan tu:"); scanf("%d",&n);
    for (int i=0; i<n; i++)
    { printf("a[%d]:",i);
      scanf("%d",&a[i]);
    }
    printf("\nMang da nhap:");
    for (i=0; i<n; i++)
        printf("%3d",a[i]);

    printf("\nMang da nhap:");
    for (i=0; i<n; i++)
        printf("%3d",*(a+i));

    printf("\nMang da nhap:");
    for (i=0; i<n; i++)
        printf("%3d",p[i]);

    printf("\nMang da nhap:");
    for (i=0; i<n; i++)
        printf("%3d",*(p+i));

    getch();
}
```



```
Nhap so luong phan tu:5
a[0]:1
a[1]:2
a[2]:3
a[3]:4
a[4]:5

Mang da nhap: 1 2 3 4 5
Mang da nhap: 1 2 3 4 5
Mang da nhap: 1 2 3 4 5
Mang da nhap: 1 2 3 4 5_
```

4 kết quả như nhau

Con trỏ và xâu ký tự

- Xem 2 cách khai báo sau:

`char hoten1[30];` // *Khai báo biến hoten1 là 1 xâu ký tự có tối đa 30 ký tự*

`char *hoten2;` // *Khai báo biến con trỏ hoten2 kiểu char*

- Nếu ta sử dụng các phép gán sau:

`hoten1 = "Nguyen Van A";` // *→ không chấp nhận*

`hoten2 = "Nguyen Van B";` // *→ chấp nhận*

Bài tập

Example 1. Chương trình tạo một mảng số nguyên ngẫu nhiên có n phần tử, giá trị các phần tử nằm trong khoảng từ -10 đến 10. Thực hiện các chức năng sau:

- In các phần tử ra màn hình
- Thêm 1 phần tử vào đầu mảng
- Thêm 1 phần tử vào cuối mảng
- Thêm 1 phần tử vào vị trí bất kỳ
- Xóa phần tử đầu tiên trong mảng có giá trị bằng x

Giải Example 1

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int randomInRange(int, int);
void printArray(int[], int);
int findIndexOfX(int[], int, int);
int unshiftArray(int[], int, int);
int pushArray(int[], int, int);
int insertAtRandomIndex(int[], int, int);
int deleteAtIndex(int[], int, int);
int main(void)
{
    srand(time(0));
    int n, arr[100];
    do
    {
        printf("Nhap so phan tu cua mang: ");
        scanf("%d", &n);
        getchar();
        if (n <= 0)
        {
            printf("Nhap sai, so phan tu cua mang  
phai > 0!\n");
        }
    } while (n <= 0);
```

```
for (int i = 0; i < n; i++)
{
    arr[i] = randomInRange(-10, 10);
}
printf("- Cac phan tu cua mang la: ");
printArray(arr, n);
printf("\n");
printf("- Them 1 phan tu vao dau mang\n");
int a;
printf("\tNhap phan tu can them: ");
scanf("%d", &a);
getchar();
n = unshiftArray(arr, n, a);
printf("\tCac phan tu cua mang la: ");
printArray(arr, n);
printf("\n");
printf("- Them 1 phan tu vao cuoi mang\n");
int b;
printf("\tNhap phan tu can them: ");
scanf("%d", &b);
getchar();
n = pushArray(arr, n, b);
printf("\tCac phan tu cua mang la: ");
printArray(arr, n);
printf("\n");
```

```
printf("- Them 1 phan tu vao vi tri bat ki trong  
mang\n");
int c;
printf("\tNhap phan tu can them: ");
scanf("%d", &c);
getchar();
n = insertAtRandomIndex(arr, n, c);
printf("\tCac phan tu cua mang la: ");
printArray(arr, n);
printf("\n");
printf("- Xoa phan tu dau tien trong mang co gia tri  
bang x\n");
int x;
printf("\tNhap x can tim: ");
scanf("%d", &x);
getchar();
int indexToDelete = findIndexOfX(arr, n, x);
if (indexToDelete == -1)
{
    printf("\t%d không có trong mảng đã cho\n", x);
}
else
{
    n = deleteAtIndex(arr, n, indexToDelete);
}
printf("\tCac phan tu cua mang la: ");
printArray(arr, n);
printf("\n");
printf("Bam ENTER de tat chuong trinh...\n");
getchar();
return 0;
}
```

Giải Example 1 (tiếp theo)

```
int randomInRange(int min, int max)
{
    return (rand() % (max - min + 1)) + min;
}

void printArray(int arr[], int n)
{
    for (int i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }
}

int unshiftArray(int arr[], int n, int new)
{
    for (int i = n; i > 0; i--)
    {
        arr[i] = arr[i - 1];
    }
    arr[0] = new;
    return n + 1;
}

int pushArray(int arr[], int n, int new)
{
    arr[n] = new;
    return n + 1;
}
```

```
int insertAtRandomIndex(int arr[], int n, int new)
{
    int indexToInsert = randomInRange(0, n);
    for (int i = n + 1; i > indexToInsert; i--)
    {
        arr[i] = arr[i - 1];
    }
    arr[indexToInsert] = new;
    return n + 1;
}

int deleteAtIndex(int arr[], int n, int index)
{
    for (int i = index; i < n - 1; i++)
    {
        arr[i] = arr[i + 1];
    }

    return n - 1;
}
```

```
int findIndexOfX(int arr[], int n, int x)
{
    for (int i = 0; i < n; i++)
    {
        if (arr[i] == x)
        {
            return i;
        }
    }
    return -1;
}
```

Giải Example 1 (sử dụng con trỏ)

```
#include <stdio.h>
#include <conio.h>
#include <time.h>
#include <stdlib.h>
#define MAX 100

// Khai bao ham nguyen mau (Prototype)
void RandomArr(int [], int);
void OutputArr(int [], int);
void Insert( int *, int *, int *, int);
void Delete(int *, int *,int);

int main() {

    int *arr,n,i;

    arr = (int*) malloc(n* sizeof(int));

    int val;

    do {

        printf("Nhap N so phan tu cua mang:\n");

        scanf("%d",&n);

        if (n<=0) printf("Khong hop le!\n");

    } while (n <=0);
```

```
printf("1. Cac phan tu cua mang:\n ");

RandomArr(arr, n);

OutputArr(arr, n);

printf("\n2. Them phan tu dau mang:\n");

Insert(arr,&n,&val,0);

printf("\n Mang sau khi chen:\n");

OutputArr(arr, n);

printf("\n3. Them phan tu cuoi mang:\n");

Insert(arr,&n,&val,n);

OutputArr(arr,n);

printf("\n4. Them phan tu vao vi tri bat ky.\n");

srand(time(0));

int p = rand() %(n);

Insert(arr,&n,&val,p);

OutputArr(arr,n);
```

```
printf("\n 5.Xoa phan tu dau tien cua mang.\n");

Delete(arr,&n,0);

OutputArr(arr,n);

free(arr);

return 0;

}

// Thiep lap cac ham

void RandomArr(int arr[], int n) {

    // Ham nhap mang ngau nhien

    srand(time(0));

    int i;

    for (i=0; i<n; i++) {

        arr[i] = -10 + rand() %21;

    }

}
```


Giải Example 1 (sử dụng con trỏ)

```
void OutputArr(int arr[], int n) {  
  
    // Ham xuất mảng  
    int i;  
    for(i=0; i<n; i++) {  
        printf("%4d ", arr[i]);  
    }  
}  
  
void Insert( int *arr, int *n, int *val, int p)  
{  
    //Ham them phan tu vao mang  
    printf("Nhap gia tri phan tu can chen:\n");  
    scanf("%d",&val);  
    int i;  
    if (*n >= MAX) {  
        return;  
    }  
    if (p < 0 ) {  
        p = 0;  
    }  
    else if (p > *n) {  
        p = *n;  
    }  
    for (i = *n-1; i >= p; i --) {  
        *(arr+i+1) = *(arr+i);  
    }  
    (*n)++;  
    *(arr+p) = *val;  
}
```

```
void Delete( int *arr, int *n, int p)  
{  
  
    //Ham xoa phan tu cua mang  
    int i;  
    if (n <= 0) {  
        return;  
    }  
  
    if (p < 0) {  
        p = 0;  
    }  
    else if (p > *n) {  
        p = *n-1;  
    }  
    for (i = p; i < *n-1 ; i ++ ) {  
        *(arr+i) = *(arr+i+1);  
    }  
    *n-- ;  
}
```

Bài tập

1/ Viết chương trình nhập tên của 1 sinh viên, in ra tên là chữ viết hoa và cắt bỏ tất cả khoảng trắng. (sử dụng con trỏ).

2/ Sử dụng con trỏ để viết chương trình thực hiện các công việc sau trên một mảng số nguyên: (mỗi công việc là 1 hàm)

- Nhập mảng
- Xuất mảng
- Sắp xếp mảng
- Tìm phần tử âm lớn nhất của mảng
- Tìm phần tử dương bé nhất của mảng
- Tìm những phần tử là số nguyên tố

CHUỖI - STRING

BÀI 10

Khái niệm

- Chuỗi là một mảng các ký tự kết thúc bằng ký tự NULL ('\0').
- Ký tự '\0' NULL tự động được thêm vào trong chuỗi.
- Mỗi ký tự trong chuỗi được lưu trữ theo định dạng bảng mã ASCII

'H'	'e'	'l'	'l'	'o'	NULL
01001000	01100101	01101100	01101100	01101111	00000000
72, 48h	101, 65h	108, 6Ch	108, 6Ch	111, 6Fh	00000000

Khai báo chuỗi

char Tên biến [độ dài của chuỗi]

Ví dụ :

```
char hoten[30];
```

```
char diachi[50];
```

hoặc

```
char *hoten;
```

```
char *diachi;
```

Nhập xuất chuỗi

Giả sử khai báo: `char S[80];`

- **Nhập chuỗi**

`scanf("%s",&S);`

hoặc `gets(S);`

- **Xuất chuỗi**

`printf(S);`

`printf("%s",S);`

hoặc `puts(S);`

- Hàm **scanf** không nhập ký tự khoảng trắng
- Để nhập chuỗi có khoảng trắng ta dùng hàm **gets**
- Nếu vùng đệm keyboard có dữ liệu thì dữ liệu sẽ chuyển vào biến.
- Để nhập một chuỗi mới chúng ta phải xóa bộ đệm keyboard bằng hàm **fflush(stdin)**

Nhập chuỗi có khoảng trắng với hàm scanf()

Sử dụng “[^\\n]” thay vì “%s”

```
char S[30];
```

```
scanf("%s",&S);
```

```
➔ scanf("%[^\\n]",&S);
```

Reads all characters until the newline ('\\n'),

Nhập chuỗi với hàm scanf()

Chuỗi định dạng	Mô tả
%[abcd]	Cho phép nhận các ký tự a, b, c, và d
%[^abcd]	Cho phép nhập tất cả ký tự ngoại trừ a, b, c, và d
%[0-9]	Cho phép nhập các ký số từ 0 đến 9
%[A-Z]	Cho phép nhập các ký tự hoa từ A→Z
%[0-9A-Za-z]	Cho phép nhập các ký số từ 0→9, ký tự A→Z, a→z
%[A-FT-Z]	Cho phép nhập các ký tự hoa từ A→F và từ T→Z

Nhập/xuất chuỗi

```
#include<stdio.h>
#include<conio.h>
void main()
{
    char S[20];
    printf("Nhap chuoai bang ham scanf ");
    scanf("%s", &S);
    printf("\nChuoai vua nhap: %s", S);
    printf("\nNhap chuoai bang ham gets");
    gets(S);
    printf("\nChuoai vua nhap:");
    puts(S);
    getch();
}
```

puts(S)
tương đương với
printf("%s\n", S)

Truy xuất từng ký tự của chuỗi

- Chuỗi là một mảng các ký tự. Vì thế chúng ta có thể truy xuất vào từng ký tự của nó bằng chỉ số.

```
#include<stdio.h>
#include<conio.h>
#include<ctype.h>
void main()
{
    char S[20];
    printf("Nhap chuoi: ");
    gets(S);
    int i=0;
    while(S[i]!=NULL)
    {
        S[i] = toupper(S[i]);
        i++;
    }
    printf("Chuoi vua nhap duoc viet hoa:");
    puts(S);
    getch();
}
```



Nhập 1 chuỗi
rồi đổi chuỗi
đó ra chữ hoa

Một số hàm về chuỗi

- **strlen(s)**

Cho biết độ dài của chuỗi s

Ví dụ : $n = \text{strlen}(\text{"abcd"}) \rightarrow n = 4$

- **strcmp(s1, s2)**

So sánh chuỗi s1 với s2.

Ví dụ : $n = \text{strcmp}(s1, s2);$

Nếu $n > 0$ thì $s1 > s2$

Nếu $n = 0$ thì $s1 = s2$

Nếu $n < 0$ thì $s1 < s2$

(So sánh dựa vào bảng mã ASCII)

Một số hàm về chuỗi

- **strcpy(đích, nguồn)**

Chép chuỗi nguồn vào chuỗi đích (gán chuỗi)

Ví dụ : `strcpy(ten, "Nguyen Van A");` → `ten = Nguyen Van A`

- **strcat(s1, s2)**

Nối chuỗi s2 vào sau s1.

Ví dụ : `s1 = "abcde"; s2 = "ghk"`

`strcat(s1, s2);` → `s1=abcdeghk`

Một số hàm về ký tự

- **toupper(ch)**
Đổi ký tự **ch** thành chữ in hoa
- **isupper(ch)**
Kiểm tra ký tự **ch** có phải là chữ in hoa không?
- **tolower(ch)**
Đổi ký tự **ch** thành chữ in thường
- **islower(ch)**
Kiểm tra ký tự **ch** có phải là chữ thường không
- **isdigit(ch)**
Kiểm tra ký tự **ch** có phải là ký tự số hay không
- **isalpha(ch)**
Kiểm tra ký tự **ch** có phải là ký tự chữ hay không

Bài tập

- Viết chương trình nhập họ tên của một người rồi in ra màn hình
- Viết chương trình nhập vào một chuỗi, rồi in các ký tự của chuỗi ra màn hình (mỗi ký tự là 1 hàng)
- Viết chương trình nhập một chuỗi, hãy cho biết số ký tự chữ (letter), số ký tự số (digit), số khoảng trắng, số ký tự khác.
- Nhập vào một chuỗi, hãy in ngược chuỗi đã nhập
- Nhập họ tên sinh viên rồi in ra tên viết tắt của sinh viên đó. Ví dụ nhập : Nguyen Thi Lan Anh → NTLA
- Nhập họ tên của giáo viên rồi in ra địa chỉ email của giáo viên đó. Ví dụ nhập : Nguyen Van Son → son.nguyenvan@hoasen.edu.vn
- Viết chương trình nhập họ tên, mã SV. In ra địa chỉ email của mình như trường Đại học Hoa sen đã cấp cho mình.

HỎI ĐÁP

